

The confusion arises because the recursive case has one line ❶, half of which is executed before the recursive call and half of which takes place after the recursive call returns. We aren't used to the idea of only half of a line of code executing at a time.

The first half is `factorial(number - 1)`. This involves calculating `number - 1` and making a recursive function, causing a new frame object to be pushed to the call stack. This happens before the recursive call is made.

The next time the code runs with the old frame object is after `factorial(number - 1)` has returned. When `factorial(5)` is called, `factorial(number - 1)` will be `factorial(4)`, which returns 24. This is when the second half of the line runs. The `return number * factorial(number - 1)` now looks like `return 5 * 24`, which is why `factorial(5)` returns 120.

[Figure 2-1](#) tracks the state of the call stack as frame objects are pushed (which happens as recursive function calls are made) and frame objects are popped (as recursive function calls return). Notice that the multiplication happens after the recursive calls are made, not before.

When the original function call to `factorial()` returns, it returns the calculated factorial.

Why the Recursive Factorial Algorithm Is Terrible

The recursive implementation for calculating factorials has a critical weakness. Calculating the factorial of 5 requires five recursive function calls. This means five frame objects are placed on the call stack before the base case is reached. This doesn't scale.

If you want to calculate the factorial of 1,001, the recursive `factorial()` function must make 1,001 recursive function calls. However, your program is likely to cause a stack overflow before it can finish, because making so many function calls without returning would exceed the maximum call stack size of the interpreter. This is terrible; you would never want to use a recursive factorial function in real-world code.